

Компьютерная графика

Лекция 2: Асинхронность, растеризация, графический конвейер, шейдеры, интерполяция, аффинные преобразования

2025

- GPU – отдельное периферийное устройство

- GPU – отдельное периферийное устройство
 - К нему нет прямого доступа

- GPU – отдельное периферийное устройство
 - К нему нет прямого доступа
 - Одновременно используется несколькими процессами

- GPU – отдельное периферийное устройство
 - К нему нет прямого доступа
 - Одновременно используется несколькими процессами
 - Одновременно выполняет несколько задач

- GPU – отдельное периферийное устройство
 - К нему нет прямого доступа
 - Одновременно используется несколькими процессами
 - Одновременно выполняет несколько задач
- Все команды оно выполняет *асинхронно*

- SIMD – Single Instruction, Multiple Data

- SIMD – Single Instruction, Multiple Data
- GPU – как SIMD-расширения процессоров (SSE, AVX), но на более крупном масштабе: сотни/тысячи одновременных вычислений

Асинхронность GPU и SIMD

- SIMD – Single Instruction, Multiple Data
- GPU – как SIMD-расширения процессоров (SSE, AVX), но на более крупном масштабе: сотни/тысячи одновременных вычислений
- Для производительности вычислительные ядра организованы в группы (warps/wavefronts/SM) по 32/64 элемента

Асинхронность GPU и SIMD

- SIMD – Single Instruction, Multiple Data
- GPU – как SIMD-расширения процессоров (SSE, AVX), но на более крупном масштабе: сотни/тысячи одновременных вычислений
- Для производительности вычислительные ядра организованы в группы (warps/wavefronts/SM) по 32/64 элемента
- Все ядра одного wavefront'а выполняют одну и ту же операцию (но над разными данными)

Асинхронность GPU и SIMD

- SIMD – Single Instruction, Multiple Data
- GPU – как SIMD-расширения процессоров (SSE, AVX), но на более крупном масштабе: сотни/тысячи одновременных вычислений
- Для производительности вычислительные ядра организованы в группы (warps/wavefronts/SM) по 32/64 элемента
- Все ядра одного wavefront'a выполняют одну и ту же операцию (но над разными данными)
- Если мы рисуем только один треугольник, под него всё равно выделяется целый wavefront и большинство его ядер делают бесполезную работу

Асинхронность GPU и SIMD

- SIMD – Single Instruction, Multiple Data
- GPU – как SIMD-расширения процессоров (SSE, AVX), но на более крупном масштабе: сотни/тысячи одновременных вычислений
- Для производительности вычислительные ядра организованы в группы (warps/wavefronts/SM) по 32/64 элемента
- Все ядра одного wavefront'a выполняют одну и ту же операцию (но над разными данными)
- Если мы рисуем только один треугольник, под него всё равно выделяется целый wavefront и большинство его ядер делают бесполезную работу
- Максимизация загрузки wavefront'ов – важная часть низкоуровневой оптимизации GPU

- Старые графические API (напр. OpenGL) прятали это в реализацию драйвера

- Старые графические API (напр. OpenGL) прятали это в реализацию драйвера
- Современные API (напр. WebGPU) явно отражают это в своём интерфейсе

- Старые графические API (напр. OpenGL) прятали это в реализацию драйвера
- Современные API (напр. WebGPU) явно отражают это в своём интерфейсе
- - Нет операции “нарисовать объект”
- + Есть операция “записать последовательность команд и отправить в очередь на исполнение”

- `CommandEncoder` – объект, позволяющий записать набор команд для исполнения на GPU

Queue, command encoder, command buffer

- `CommandEncoder` – объект, позволяющий записать набор команд для исполнения на GPU
- `encoder.finish()` -> `CommandBuffer` – записанный буфер с командами в понятном драйверу/GPU виде

Queue, command encoder, command buffer

- `CommandEncoder` – объект, позволяющий записать набор команд для исполнения на GPU
- `encoder.finish()` -> `CommandBuffer` – записанный буфер с командами в понятном драйверу/GPU виде
- `queue.submit(buffers)` – отправить команды на исполнение (принимает массив `CommandBuffer`'ов)

- Растеризация – превращение геометрического примитива (точки, линии, треугольника, прямоугольника, круга, и т.д.) в набор соответствующих ему пикселей на экране/изображении

- Растеризация – превращение геометрического примитива (точки, линии, треугольника, прямоугольника, круга, и т.д.) в набор соответствующих ему пикселей на экране/изображении
- Превращение векторных данных в растровые

- Растеризация – превращение геометрического примитива (точки, линии, треугольника, прямоугольника, круга, и т.д.) в набор соответствующих ему пикселей на экране/изображении
- Превращение векторных данных в растровые
- За нас её делает GPU!

- Растеризация – превращение геометрического примитива (точки, линии, треугольника, прямоугольника, круга, и т.д.) в набор соответствующих ему пикселей на экране/изображении
- Превращение векторных данных в растровые
- За нас её делает GPU!
- Некоторые современные графические движки (Unreal 5 Nanite) делают растеризацию сами с помощью compute шейдеров

Почему растеризация?

- Исторически было очень много разных способов рисовать трёхмерные сцены; см. Sutherland et al - A Characterization of Ten Hidden-Surface Algorithms

Почему растеризация?

- Исторически было очень много разных способов рисовать трёхмерные сцены; см. Sutherland et al - A Characterization of Ten Hidden-Surface Algorithms
- В современности есть два основных алгоритма:

Почему растеризация?

- Исторически было очень много разных способов рисовать трёхмерные сцены; см. Sutherland et al - A Characterization of Ten Hidden-Surface Algorithms
- В современности есть два основных алгоритма:
 - **Растеризация:** какие пиксели занимает конкретный объект

Почему растеризация?

- Исторически было очень много разных способов рисовать трёхмерные сцены; см. Sutherland et al - A Characterization of Ten Hidden-Surface Algorithms
- В современности есть два основных алгоритма:
 - **Растеризация**: какие пиксели занимает конкретный объект
 - **Трассировка лучей**: какие объекты видит конкретный пиксель

Почему растеризация?

- Особенности растеризации:
 - + Быстрее (зависит от сцены)

Почему растеризация?

- Особенности растеризации:
 - + Быстрее (зависит от сцены)
 - + Легче параллелится

Почему растеризация?

- Особенности растеризации:
 - + Быстрее (зависит от сцены)
 - + Легче параллелится
 - + Лучше ложится на железо

Почему растеризация?

- Особенности растеризации:
 - + Быстрее (зависит от сцены)
 - + Легче параллелится
 - + Лучше ложится на железо
 - + Лучше работает с памятью (data locality)

Почему растеризация?

- Особенности растеризации:
 - + Быстрее (зависит от сцены)
 - + Легче параллелится
 - + Лучше ложится на железо
 - + Лучше работает с памятью (data locality)
 - - Сложнее добиться фотореалистичной графики

Почему растеризация?

- Особенности растеризации:
 - + Быстрее (зависит от сцены)
 - + Легче параллелится
 - + Лучше ложится на железо
 - + Лучше работает с памятью (data locality)
 - - Сложнее добиться фотореалистичной графики
- Особенности трассировки лучей:

Почему растеризация?

- Особенности растеризации:
 - + Быстрее (зависит от сцены)
 - + Легче параллелится
 - + Лучше ложится на железо
 - + Лучше работает с памятью (data locality)
 - - Сложнее добиться фотореалистичной графики
- Особенности трассировки лучей:
 - Всё ровно наоборот :)

- Как растеризовать точку (x, y) ?

- Как растеризовать точку (x, y) ?

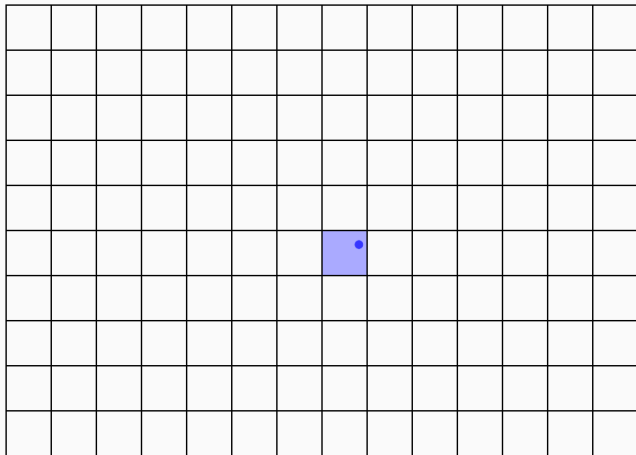
```
set_pixel(floor(x), floor(y), color);
```

- Как растеризовать точку (x, y) ?

```
set_pixel(floor(x), floor(y), color);
```

- В WebGPU: `point-list`

Растеризация: точка



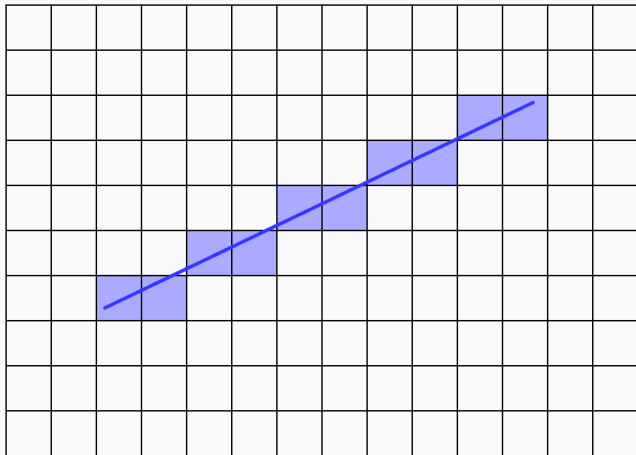
- Как растеризовать линию $(x_1, y_1) \dots (x_2, y_2)$?

- Как растеризовать линию $(x_1, y_1) \dots (x_2, y_2)$?
- Алгоритм Брезенхэма

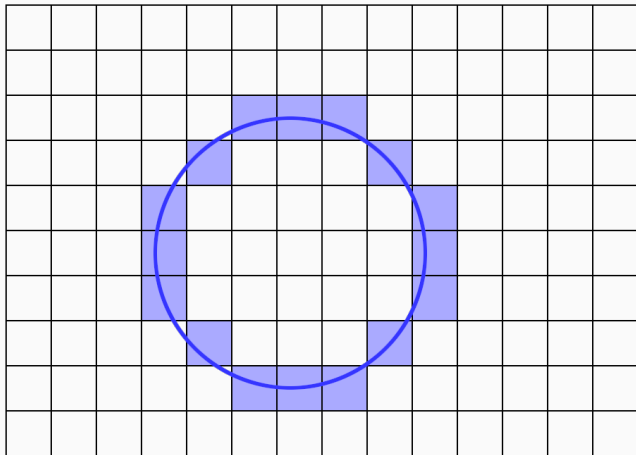
- Как растеризовать линию $(x_1, y_1) \dots (x_2, y_2)$?
- Алгоритм Брезенхэма
- Есть вариация алгоритма для рисования окружностей

- Как растеризовать линию $(x_1, y_1) \dots (x_2, y_2)$?
- Алгоритм Брезенхэма
- Есть вариация алгоритма для рисования окружностей
- В WebGPU: `line-list`, `line-strip`

Растеризация: линия



Растеризация: окружность



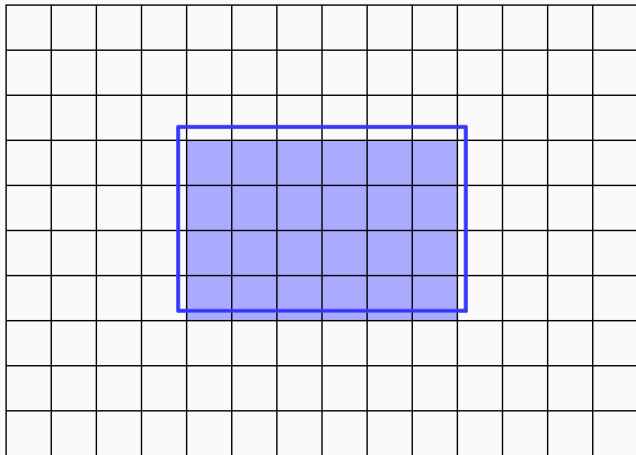
- Как растеризовать прямоугольник $[x_1 \dots x_2] \times [y_1 \dots y_2]$?

Растрезация: прямоугольник

- Как растрезировать прямоугольник $[x_1 \dots x_2] \times [y_1 \dots y_2]$?

```
for (int x = round(x_1); x < round(x_2); ++x) {  
    for (int y = round(y_1); y < round(y_2); ++y) {  
        set_pixel(x, y, color);  
    }  
}
```

Растеризация: прямоугольник



Растеризация: треугольник

- Как растеризовать треугольник с вершинами (x_1, y_1) , (x_2, y_2) , (x_3, y_3) ?

Растеризация: треугольник

- Как растеризовать треугольник с вершинами (x_1, y_1) , (x_2, y_2) , (x_3, y_3) ?
- Растеризуем ограничивающий прямоугольник, проверяя пиксели на вхождение в треугольник

Растеризация: треугольник

- Как растеризовать треугольник с вершинами (x_1, y_1) , (x_2, y_2) , (x_3, y_3) ?
- Растеризуем ограничивающий прямоугольник, проверяя пиксели на входжение в треугольник

```
int xmin = min(floor(x_1), floor(x_2), floor(x_3));
int xmax = max( ceil(x_1),  ceil(x_2),  ceil(x_3));

int ymin = min(floor(y_1), floor(y_2), floor(y_3));
int ymax = max( ceil(y_1),  ceil(y_2),  ceil(y_3));

for (int x = xmin; x <= xmax; ++x) {
    for (int y = ymin; y <= ymax; ++y) {
        if (inside_triangle(x, y, ...)) {
            set_pixel(x, y, color);
        }
    }
}
```

Растеризация: треугольник

- Как растеризовать треугольник с вершинами $(x_1, y_1), (x_2, y_2), (x_3, y_3)$?
- Растеризуем ограничивающий прямоугольник, проверяя пиксели на вхождение в треугольник

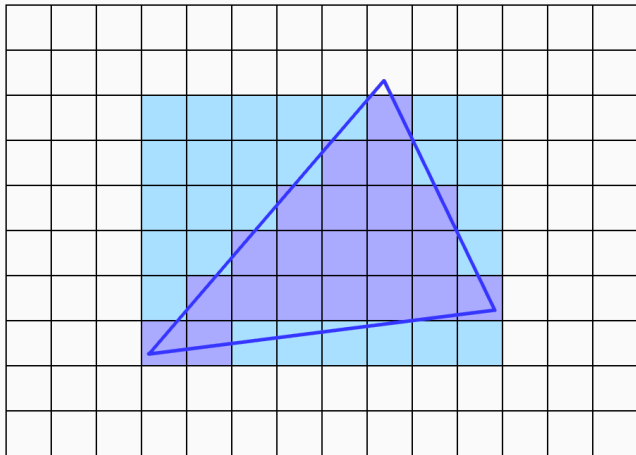
```
int xmin = min(floor(x_1), floor(x_2), floor(x_3));
int xmax = max(ceil(x_1),  ceil(x_2),  ceil(x_3));

int ymin = min(floor(y_1), floor(y_2), floor(y_3));
int ymax = max(ceil(y_1),  ceil(y_2),  ceil(y_3));

for (int x = xmin; x <= xmax; ++x) {
    for (int y = ymin; y <= ymax; ++y) {
        if (inside_triangle(x, y, ...)) {
            set_pixel(x, y, color);
        }
    }
}
```

- В WebGPU: `triangle-list`, `triangle-strip`

Растеризация: треугольник



- Как растеризовать круг с центром (x_0, y_0) и радиусом R ?

Растеризация: круг

- Как растеризовать круг с центром (x_0, y_0) и радиусом R ?
- Растеризуем ограничивающий прямоугольник, проверяя пиксели на вхождение в круг

Растеризация: круг

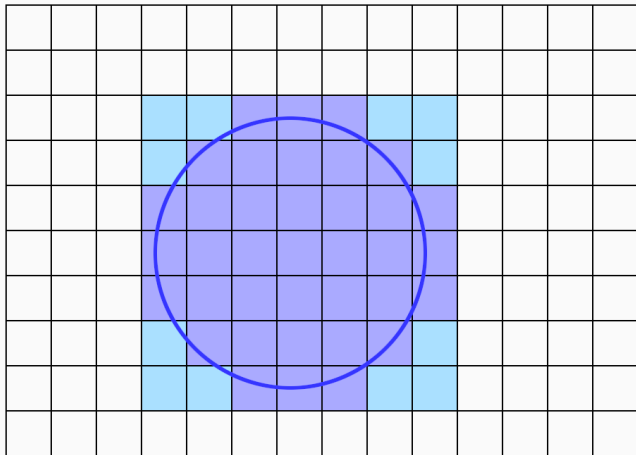
- Как растеризовать круг с центром (x_0, y_0) и радиусом R ?
- Растеризуем ограничивающий прямоугольник, проверяя пиксели на вхождение в круг

```
int xmin = floor(x_0 - R);
int xmax =  ceil(x_0 + R);

int ymin = floor(y_0 - R);
int ymax =  ceil(y_0 + R);

for (int x = xmin; x <= xmax; ++x) {
    for (int y = ymin; y <= ymax; ++y) {
        if (sqr(x - x_0) + sqr(y - y_0) <= sqr(R))
            set_pixel(x, y, color);
    }
}
```

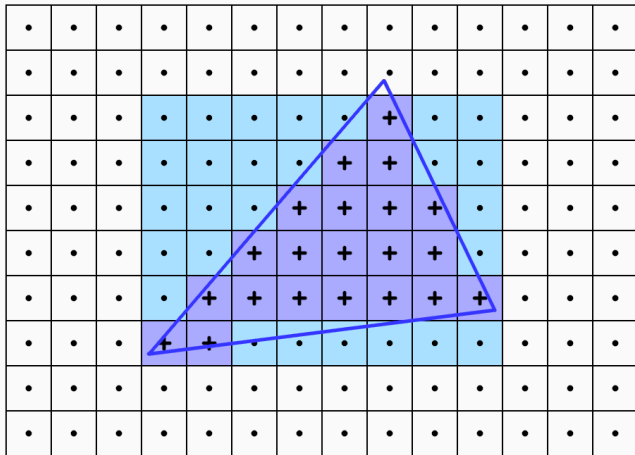
Растеризация: круг



Правила растеризации

- Пиксель растеризуется, если *центр пикселя* содержится в треугольнике

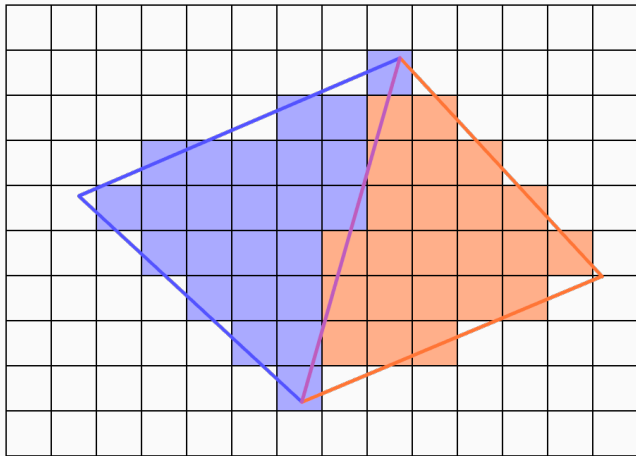
Правила растеризации



- Пиксель растеризуется, если *центр пикселя* содержится в треугольнике

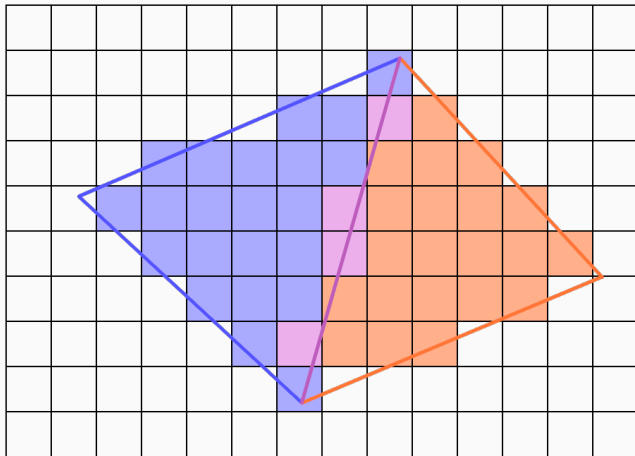
Правила растеризации

- Пиксель растеризуется, если *центр пикселя* содержится в треугольнике
- Если у двух треугольников есть общее ребро (и они не пересекаются внутренностями), то
 - Каждый пиксель будет принадлежать ровно одному треугольнику, т.е. не будет наложения
 - Ни один пиксель общего ребра не будет пропущен, т.е. не будет “дырок”



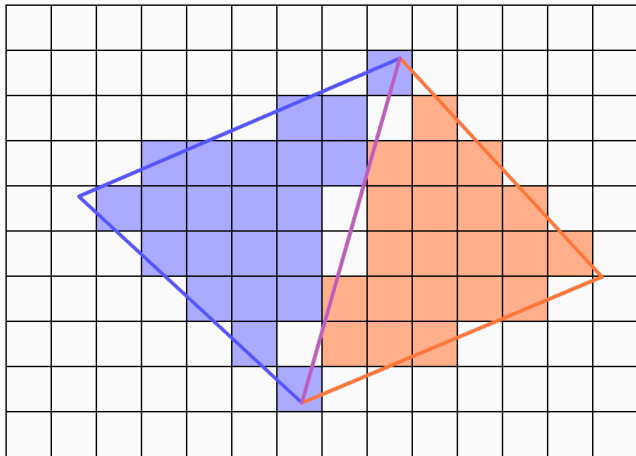
Правила растеризации

Не будет наложения пикселей:



Правила растеризации

Не будет дырок:



Правила растеризации

- Пиксель растеризуется, если *центр пикселя* содержится в треугольнике
- Если у двух треугольников есть общее ребро (и они не пересекаются внутренностями), то
 - Каждый пиксель будет принадлежать ровно одному треугольнику, т.е. не будет наложения
 - Ни один пиксель общего ребра не будет пропущен, т.е. не будет “дырок”

Правила растеризации

- Пиксель растеризуется, если *центр пикселя* содержится в треугольнике
- Если у двух треугольников есть общее ребро (и они не пересекаются внутренностями), то
 - Каждый пиксель будет принадлежать ровно одному треугольнику, т.е. не будет наложения
 - Ни один пиксель общего ребра не будет пропущен, т.е. не будет “дырок”
- Подробнее: en.wikibooks.org/wiki/GLSL_Programming/Rasterization

- В современных графических API есть только три основных примитива для растеризации:

- В современных графических API есть только три основных примитива для растеризации:
 - Точки: `point-list`

- В современных графических API есть только три основных примитива для растеризации:
 - Точки: `point-list`
 - Линии: `line-list`, `list-strip`

Правила растеризации

- В современных графических API есть только три основных примитива для растеризации:
 - Точки: `point-list`
 - Линии: `line-list`, `list-strip`
 - Треугольники: `triangle-list`, `triangle-strip`

Правила растеризации

- В современных графических API есть только три основных примитива для растеризации:
 - Точки: `point-list`
 - Линии: `line-list`, `list-strip`
 - Треугольники: `triangle-list`, `triangle-strip`
- Точки и линии обычно используются для дебажного рендеринга или вместе с геометрическими шейдерами, *главный примитив – треугольник*

- Почему основным примитивом рисования стал *треугольник*?

- Почему основным примитивом рисования стал *треугольник*?
- Для более сложных геометрических фигур (круг, многоугольник, и т.д.):

- Почему основным примитивом рисования стал *треугольник*?
- Для более сложных геометрических фигур (круг, многоугольник, и т.д.):
 - - Сложнее алгоритм разтеризации (особенно с учётом перспективной проекции)

- Почему основным примитивом рисования стал *треугольник*?
- Для более сложных геометрических фигур (круг, многоугольник, и т.д.):
 - - Сложнее алгоритм разтеризации (особенно с учётом перспективной проекции)
 - - Сложнее задавать и интерполировать атрибуты (накладывать цвет и текстуру, вычислять освещение)

О треугольниках

Плюсы треугольника:

Плюсы треугольника:

- + Образ под действием перспективной проекции – тоже треугольник

Плюсы треугольника:

- + Образ под действием перспективной проекции – тоже треугольник
- + Есть единственный разумный способ интерполяции (линейная, с барицентрическими координатами)

Плюсы треугольника:

- + Образ под действием перспективной проекции – тоже треугольник
- + Есть единственный разумный способ интерполяции (линейная, с барицентрическими координатами)
- + Позволяет рисовать спрайты (прямоугольник – два треугольника)

Плюсы треугольника:

- + Образ под действием перспективной проекции – тоже треугольник
- + Есть единственный разумный способ интерполяции (линейная, с барицентрическими координатами)
- + Позволяет рисовать спрайты (прямоугольник – два треугольника)
- + Позволяет рисовать многоугольники (посредством триангуляции)

О треугольниках

Плюсы треугольника:

- + Образ под действием перспективной проекции – тоже треугольник
- + Есть единственный разумный способ интерполяции (линейная, с барицентрическими координатами)
- + Позволяет рисовать спрайты (прямоугольник – два треугольника)
- + Позволяет рисовать многоугольники (посредством триангуляции)
- + Позволяет рисовать линии (превращая их в тонкие многоугольники)

Плюсы треугольника:

- + Образ под действием перспективной проекции – тоже треугольник
- + Есть единственный разумный способ интерполяции (линейная, с барицентрическими координатами)
- + Позволяет рисовать спрайты (прямоугольник – два треугольника)
- + Позволяет рисовать многоугольники (посредством триангуляции)
- + Позволяет рисовать линии (превращая их в тонкие многоугольники)
- + Позволяет рисовать более сложные фигуры (аппроксимируя)

- Последовательность операций на GPU, превращающих входные данные в картинку на экране

Графический конвейер (graphics pipeline)

- Последовательность операций на GPU, превращающих входные данные в картинку на экране
- Что происходит при выполнении `render_pass.draw()` от чтения данных до появления картинки на экране

Графический конвейер (graphics pipeline)

- Последовательность операций на GPU, превращающих входные данные в картинку на экране
- Что происходит при выполнении `render_pass.draw()` от чтения данных до появления картинки на экране
- Почти не отличается для разных графических API

Основные части конвейера:

- Входной поток вершин (vertex stream)

Графический конвейер (graphics pipeline)

Основные части конвейера:

- Входной поток вершин (vertex stream)
 - Позже мы узнаем, как его задавать

Графический конвейер (graphics pipeline)

Основные части конвейера:

- Входной поток вершин (vertex stream)
 - Позже мы узнаем, как его задавать
- Вершинный шейдер: обрабатывает вершины по одной

Графический конвейер (graphics pipeline)

Основные части конвейера:

- Входной поток вершин (vertex stream)
 - Позже мы узнаем, как его задавать
- Вершинный шейдер: обрабатывает вершины по одной
- Сборка примитивов (primitive assembly) из отдельных вершин

Графический конвейер (graphics pipeline)

Основные части конвейера:

- Входной поток вершин (vertex stream)
 - Позже мы узнаем, как его задавать
- Вершинный шейдер: обрабатывает вершины по одной
- Сборка примитивов (primitive assembly) из отдельных вершин
- Преобразование в оконную систему координат (viewport transform)

Графический конвейер (graphics pipeline)

Основные части конвейера:

- Входной поток вершин (vertex stream)
 - Позже мы узнаем, как его задавать
- Вершинный шейдер: обрабатывает вершины по одной
- Сборка примитивов (primitive assembly) из отдельных вершин
- Преобразование в оконную систему координат (viewport transform)
- Растеризация примитивов

Графический конвейер (graphics pipeline)

Основные части конвейера:

- Входной поток вершин (vertex stream)
 - Позже мы узнаем, как его задавать
- Вершинный шейдер: обрабатывает вершины по одной
- Сборка примитивов (primitive assembly) из отдельных вершин
- Преобразование в оконную систему координат (viewport transform)
- Растеризация примитивов
- Пиксельный (фрагментный) шейдер: обрабатывает пиксели по одному

Графический конвейер (graphics pipeline)

- Мы пропустили много важных частей конвейера
- Будем их по чуть-чуть добавлять в течение курса
- [khronos.org/opengl/wiki/Rendering_Pipeline_Overview](https://www.khronos.org/opengl/wiki/Rendering_Pipeline_Overview)

- Конвейер – удобная абстракция, внутри GPU всё устроено сложнее

Графический конвейер (graphics pipeline)

- Конвейер – удобная абстракция, внутри GPU всё устроено сложнее
- Каждая часть конвейера выполняется сразу на большом объёме входных данных параллельно

Графический конвейер (graphics pipeline)

- Конвейер – удобная абстракция, внутри GPU всё устроено сложнее
- Каждая часть конвейера выполняется сразу на большом объёме входных данных параллельно
- Разные части конвейера могут выполняться одновременно (на разных данных)

Графический конвейер (graphics pipeline)

- Конвейер – удобная абстракция, внутри GPU всё устроено сложнее
- Каждая часть конвейера выполняется сразу на большом объёме входных данных параллельно
- Разные части конвейера могут выполняться одновременно (на разных данных)
- Разные вызовы команд рисования могут обрабатываться параллельно

- Входные данные:

- Входные данные:
 - Атрибуты вершин (мы позже узнаем, как их задавать) – свои значения для каждой вершины (позиция, цвет, нормаль)

Вершинный (vertex) шейдер

- Входные данные:
 - Атрибуты вершин (мы позже узнаем, как их задавать) – свои значения для каждой вершины (позиция, цвет, нормаль)
 - Immediates/push constants – маленький (128 байт) набор данных, вшитый в command buffer

Вершинный (vertex) шейдер

- Входные данные:
 - Атрибуты вершин (мы позже узнаем, как их задавать) – свои значения для каждой вершины (позиция, цвет, нормаль)
 - Immediates/push constants – маленький (128 байт) набор данных, вшитый в command buffer
 - Встроенные переменные, например `@builtin(vertex_index)` – полный список смотри в документации

Вершинный (vertex) шейдер

- Входные данные:
 - Атрибуты вершин (мы позже узнаем, как их задавать) – свои значения для каждой вершины (позиция, цвет, нормаль)
 - Immediates/push constants – маленький (128 байт) набор данных, вшитый в command buffer
 - Встроенные переменные, например `@builtin(vertex_index)` – полный список смотри в документации
 - Uniform'ы, текстуры, буферы – об этом позже :)

Вершинный (vertex) шейдер

- Входные данные:
 - Атрибуты вершин (мы позже узнаем, как их задавать) – свои значения для каждой вершины (позиция, цвет, нормаль)
 - Immediates/push constants – маленький (128 байт) набор данных, вшитый в command buffer
 - Встроенные переменные, например `@builtin(vertex_index)` – полный список смотри в документации
 - Uniform'ы, текстуры, буферы – об этом позже :)
- Выходные данные:
 - `@builtin(position)` `position: vec4f`

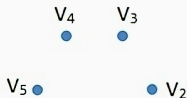
Вершинный (vertex) шейдер

- Входные данные:
 - Атрибуты вершин (мы позже узнаем, как их задавать) – свои значения для каждой вершины (позиция, цвет, нормаль)
 - Immediates/push constants – маленький (128 байт) набор данных, вшитый в command buffer
 - Встроенные переменные, например `@builtin(vertex_index)` – полный список смотри в документации
 - Uniform'ы, текстуры, буферы – об этом позже :)
- Выходные данные:
 - `@builtin(position) position: vec4f`
 - Переменные, интерполированное значение которых попадёт во фрагментный (пиксельный) шейдер:
`@location(0) color: vec4f`

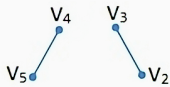
Сборка примитивов (primitive assembly)

- Превращает набор независимых вершин в *примитивы* растеризации: точки/линии/треугольники

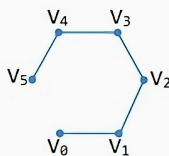
Сборка примитивов (primitive assembly)



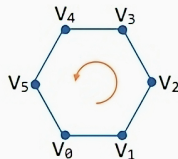
GL_POINTS



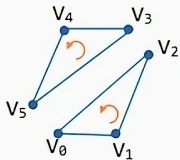
GL_LINES



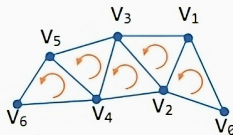
GL_LINE_STRIP



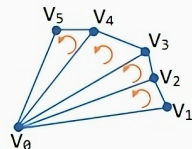
GL_LINE_LOOP



GL_TRIANGLES



GL_TRIANGLE_STRIP



GL_TRIANGLE_FAN

Сборка примитивов (primitive assembly)

- Превращает набор независимых вершин в *примитивы* растеризации: точки/линии/треугольники

Сборка примитивов (primitive assembly)

- Превращает набор независимых вершин в *примитивы* растеризации: точки/линии/треугольники
- Правила сборки зависят от выбранного типа примитива:
`point-list`, `line-list`, `line-strip`, `triangle-list`, `triangle-strip`

Viewport transform

- Преобразование в оконную (пиксельную) систему координат
- $X : [-1, 1] \rightarrow [X_{min}, X_{max}]$
- $Y : [-1, 1] \rightarrow [Y_{max}, Y_{min}]$ (-1 внизу, 1 вверху)

Viewport transform

- Преобразование в оконную (пиксельную) систему координат
- $X : [-1, 1] \rightarrow [X_{min}, X_{max}]$
- $Y : [-1, 1] \rightarrow [Y_{max}, Y_{min}]$ (-1 внизу, 1 вверху)
- Настроить: `render_pass.set_viewport()`

Viewport transform

- Преобразование в оконную (пиксельную) систему координат
- $X : [-1, 1] \rightarrow [X_{min}, X_{max}]$
- $Y : [-1, 1] \rightarrow [Y_{max}, Y_{min}]$ (-1 внизу, 1 вверху)
- Настроить: `render_pass.set_viewport()`
- По умолчанию – весь экран (точнее, весь color target)

Viewport transform

- Преобразование в оконную (пиксельную) систему координат
- $X : [-1, 1] \rightarrow [X_{min}, X_{max}]$
- $Y : [-1, 1] \rightarrow [Y_{max}, Y_{min}]$ (-1 внизу, 1 вверху)
- Настроить: `render_pass.set_viewport()`
- По умолчанию – весь экран (точнее, весь color target)
- Вне прямоугольника $[X_{min}, X_{max}] \times [Y_{min}, Y_{max}]$ *ничего не нарисует*ся: GPU обрежет все примитивы по этим границам

Фрагментный (пиксельный, fragment) шейдер

Фрагментный (пиксельный, fragment) шейдер

- Входные данные:

Фрагментный (пиксельный, fragment) шейдер

- Входные данные:
 - Проинтерполированные значения из вершинного шейдера:
`@location(0) color: vec4f`

Фрагментный (пиксельный, fragment) шейдер

- Входные данные:
 - Проинтерполированные значения из вершинного шейдера:
`@location(0) color: vec4f`
 - Immediates/push constants

Фрагментный (пиксельный, fragment) шейдер

- Входные данные:
 - Проинтерполированные значения из вершинного шейдера:
`@location(0) color: vec4f`
 - Immediates/push constants
 - Встроенные переменные, например `@builtin(position)`

Фрагментный (пиксельный, fragment) шейдер

- Входные данные:
 - Проинтерполированные значения из вершинного шейдера:
`@location(0) color: vec4f`
 - Immediates/push constants
 - Встроенные переменные, например `@builtin(position)`
 - Uniform'ы, текстуры, буферы

Фрагментный (пиксельный, fragment) шейдер

- Входные данные:
 - Проинтерполированные значения из вершинного шейдера:
`@location(0) color: vec4f`
 - Immediates/push constants
 - Встроенные переменные, например `@builtin(position)`
 - Uniform'ы, текстуры, буферы
- Выходные данные:
 - `@location(0) color: vec4f` – выходной цвет в формате RGBA, компоненты в диапазоне `[0 ... 1]`

Фрагментный (пиксельный, fragment) шейдер

- Входные данные:
 - Проинтерполированные значения из вершинного шейдера:
`@location(0) color: vec4f`
 - Immediates/push constants
 - Встроенные переменные, например `@builtin(position)`
 - Uniform'ы, текстуры, буферы
- Выходные данные:
 - `@location(0) color: vec4f` – выходной цвет в формате RGBA, компоненты в диапазоне $[0 \dots 1]$
 - Может быть несколько, об этом поговорим позже

- При рисовании нам нужно сказать, **куда** рисовать, **что** рисовать, и **как** рисовать

- При рисовании нам нужно сказать, **куда** рисовать, **что** рисовать, и **как** рисовать
- Куда (в какую текстуру) рисовать настраивается при создании `RenderPassEncoder`'а

- При рисовании нам нужно сказать, **куда** рисовать, **что** рисовать, и **как** рисовать
- Куда (в какую текстуру) рисовать настраивается при создании `RenderPassEncoder`'а
- Что рисовать настраивается через методы `RenderPassEncoder`'а (`set_vertex_buffer` и т.п.)

- При рисовании нам нужно сказать, **куда** рисовать, **что** рисовать, и **как** рисовать
- Куда (в какую текстуру) рисовать настраивается при создании `RenderPassEncoder`'а
- Что рисовать настраивается через методы `RenderPassEncoder`'а (`set_vertex_buffer` и т.п.)
- Как рисовать настраивается через `render_pass_encoder.set_pipeline()`

- Часть настроек конвейера – тяжёлые и сложные, с большим объёмом валидации: настройки атрибутов вершин, используемые шейдеры, тип примитива, и т.п.

- Часть настроек конвейера – тяжёлые и сложные, с большим объёмом валидации: настройки атрибутов вершин, используемые шейдеры, тип примитива, и т.п.
- Часть настроек – лёгкие, часто описываются парой чисел: viewport, scissor, blend constant, stencil reference

Render pipeline

- Часть настроек конвейера – тяжёлые и сложные, с большим объёмом валидации: настройки атрибутов вершин, используемые шейдеры, тип примитива, и т.п.
- Часть настроек – лёгкие, часто описываются парой чисел: viewport, scissor, blend constant, stencil reference
- Тяжёлые настройки хранит особый объект – **RenderPipeline**

- Часть настроек конвейера – тяжёлые и сложные, с большим объёмом валидации: настройки атрибутов вершин, используемые шейдеры, тип примитива, и т.п.
- Часть настроек – лёгкие, часто описываются парой чисел: viewport, scissor, blend constant, stencil reference
- Тяжёлые настройки хранит особый объект – **RenderPipeline**
- Лёгкие настройки можно менять на лету через **RenderPassEncoder**

- OpenGL, Vulkan: GLSL (GL Shading Language)

- OpenGL, Vulkan: GLSL (GL Shading Language)
- DirectX: HLSL (High-Level Shading Language)

- OpenGL, Vulkan: GLSL (GL Shading Language)
- DirectX: HLSL (High-Level Shading Language)
- DirectX (до 2012): Cg (C for Graphics), deprecated

- OpenGL, Vulkan: GLSL (GL Shading Language)
- DirectX: HLSL (High-Level Shading Language)
- DirectX (до 2012): Cg (C for Graphics), deprecated
- WebGPU: WGSL (WebGpu Shading Language)

- OpenGL, Vulkan: GLSL (GL Shading Language)
- DirectX: HLSL (High-Level Shading Language)
- DirectX (до 2012): Cg (C for Graphics), deprecated
- WebGPU: WGSL (WebGpu Shading Language)
- Slang – язык, не привязанный к API

- OpenGL, Vulkan: GLSL (GL Shading Language)
- DirectX: HLSL (High-Level Shading Language)
- DirectX (до 2012): Cg (C for Graphics), deprecated
- WebGPU: WGSL (WebGpu Shading Language)
- Slang – язык, не привязанный к API
- И другие: en.wikipedia.org/wiki/Shading_language

- Похож на Rust

- Похож на Rust
- Типы данных:

- Похож на Rust
- Типы данных:
 - Скалярные: `bool`, `i32`, `u32`, `f32`

- Похож на Rust
- Типы данных:
 - Скалярные: `bool`, `i32`, `u32`, `f32`
 - Векторные: `vec2<T>`, `vec3<T>`, `vec4<T>`

- Похож на Rust
- Типы данных:
 - Скалярные: `bool`, `i32`, `u32`, `f32`
 - Векторные: `vec2<T>`, `vec3<T>`, `vec4<T>`
 - Векторные алиасы: `vec2i`, `vec3f`, ...

- Похож на Rust
- Типы данных:
 - Скалярные: `bool`, `i32`, `u32`, `f32`
 - Векторные: `vec2<T>`, `vec3<T>`, `vec4<T>`
 - Векторные алиасы: `vec2i`, `vec3f`, ...
 - Матричные: `mat2x4<T>`, `mat4x4f`, ...

- Похож на Rust
- Типы данных:
 - Скалярные: `bool`, `i32`, `u32`, `f32`
 - Векторные: `vec2<T>`, `vec3<T>`, `vec4<T>`
 - Векторные алиасы: `vec2i`, `vec3f`, ...
 - Матричные: `mat2x4<T>`, `mat4x4f`, ...
 - **N.B.:** `mat2x4` это матрица 4×2 , т.е. с 4 строками и 2 столбцами!

- Конструкторы векторов:

- Конструкторы векторов:
 - `vec3f(x, y, z)`

- Конструкторы векторов:
 - `vec3f(x, y, z)`
 - `vec3f(x) == vec3f(x, x, x)`

- Конструкторы векторов:
 - `vec3f(x, y, z)`
 - `vec3f(x) == vec3f(x, x, x)`
 - `vec3f(vec2f(x, y), z) == vec3f(x, vec2f(y, z))`

- Конструкторы векторов:
 - `vec3f(x, y, z)`
 - `vec3f(x) == vec3f(x, x, x)`
 - `vec3f(vec2f(x, y), z) == vec3f(x, vec2f(y, z))`
- Подробнее: google.github.io/tour-of-wgsl/types

Язык описания шейдеров WGSL

- Есть стандартные операции: $+$, $-$, $*$, $/$, $\%$, $<$, $==$, ...

Язык описания шейдеров WGSL

- Есть стандартные операции: $+$, $-$, $*$, $/$, $\%$, $<$, $==$, ...
 - Для векторов и матриц применяются покомпонентно

Язык описания шейдеров WGSL

- Есть стандартные операции: $+$, $-$, $*$, $/$, $\%$, $<$, $==$, ...
 - Для векторов и матриц применяются покомпонентно
 - Исключение: умножение двух матриц это умножение в смысле алгебры

Язык описания шейдеров WGSL

- Есть стандартные операции: $+$, $-$, $*$, $/$, $\%$, $<$, $==$, ...
 - Для векторов и матриц применяются покомпонентно
 - Исключение: умножение двух матриц это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор слева это умножение в смысле алгебры

Язык описания шейдеров WGSL

- Есть стандартные операции: $+$, $-$, $*$, $/$, $\%$, $<$, $==$, ...
 - Для векторов и матриц применяются покомпонентно
 - Исключение: умножение двух матриц это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор слева это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор справа это умножение в смысле алгебры на транспонированную матрицу

Язык описания шейдеров WGSL

- Есть стандартные операции: $+$, $-$, $*$, $/$, $\%$, $<$, $==$, ...
 - Для векторов и матриц применяются покомпонентно
 - Исключение: умножение двух матриц это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор слева это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор справа это умножение в смысле алгебры на транспонированную матрицу
- Доступ к координатам векторов: $a.x = b.y$

Язык описания шейдеров WGSL

- Есть стандартные операции: `+`, `-`, `*`, `/`, `%`, `<`, `==`, ...
 - Для векторов и матриц применяются покомпонентно
 - Исключение: умножение двух матриц это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор слева это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор справа это умножение в смысле алгебры на транспонированную матрицу
- Доступ к координатам векторов: `a.x` `=` `b.y`
- Swizzling: `a.xxzy` `==` `vec4f(a.x, a.x, a.z, a.y)`

Язык описания шейдеров WGSL

- Есть стандартные операции: `+`, `-`, `*`, `/`, `%`, `<`, `==`, ...
 - Для векторов и матриц применяются покомпонентно
 - Исключение: умножение двух матриц это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор слева это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор справа это умножение в смысле алгебры на транспонированную матрицу
- Доступ к координатам векторов: `a.x == b.y`
- Swizzling: `a.xxzy == vec4f(a.x, a.x, a.z, a.y)`
- Доступ к элементам матриц: `m[column][row]`

Язык описания шейдеров WGSL

- Есть стандартные операции: `+`, `-`, `*`, `/`, `%`, `<`, `==`, ...
 - Для векторов и матриц применяются покомпонентно
 - Исключение: умножение двух матриц это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор слева это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор справа это умножение в смысле алгебры на транспонированную матрицу
- Доступ к координатам векторов: `a.x` `=` `b.y`
- Swizzling: `a.xxzy` `==` `vec4f(a.x, a.x, a.z, a.y)`
- Доступ к элементам матриц: `m[column][row]`
- Есть полезные математические функции: `pow`, `sin`, `cos`, ...

Язык описания шейдеров WGSL

- Есть стандартные операции: `+`, `-`, `*`, `/`, `%`, `<`, `==`, ...
 - Для векторов и матриц применяются покомпонентно
 - Исключение: умножение двух матриц это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор слева это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор справа это умножение в смысле алгебры на транспонированную матрицу
- Доступ к координатам векторов: `a.x` `=` `b.y`
- Swizzling: `a.xxzy` `==` `vec4f(a.x, a.x, a.z, a.y)`
- Доступ к элементам матриц: `m[column][row]`
- Есть полезные математические функции: `pow`, `sin`, `cos`, ...
 - Для векторов и матриц применяются покомпонентно

Язык описания шейдеров WGSL

- Есть стандартные операции: `+`, `-`, `*`, `/`, `%`, `<`, `==`, ...
 - Для векторов и матриц применяются покомпонентно
 - Исключение: умножение двух матриц это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор слева это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор справа это умножение в смысле алгебры на транспонированную матрицу
- Доступ к координатам векторов: `a.x = b.y`
- Swizzling: `a.xxzy == vec4f(a.x, a.x, a.z, a.y)`
- Доступ к элементам матриц: `m[column][row]`
- Есть полезные математические функции: `pow`, `sin`, `cos`, ...
 - Для векторов и матриц применяются покомпонентно
- Есть математические функции векторной алгебры: `dot`, `cross`, `length`, `normalize`, ...

Язык описания шейдеров WGSL

- Есть стандартные операции: `+`, `-`, `*`, `/`, `%`, `<`, `==`, ...
 - Для векторов и матриц применяются покомпонентно
 - Исключение: умножение двух матриц это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор слева это умножение в смысле алгебры
 - Исключение: умножение матрицы на вектор справа это умножение в смысле алгебры на транспонированную матрицу
- Доступ к координатам векторов: `a.x = b.y`
- Swizzling: `a.xxzy == vec4f(a.x, a.x, a.z, a.y)`
- Доступ к элементам матриц: `m[column][row]`
- Есть полезные математические функции: `pow`, `sin`, `cos`, ...
 - Для векторов и матриц применяются покомпонентно
- Есть математические функции векторной алгебры: `dot`, `cross`, `length`, `normalize`, ...
- Линейная и нелинейная интерполяция: `mix`, `smoothstep`

- Есть массивы:

```
let a = array<f32, 5>(0.0, 1.0, 2.0, 3.0, 4.0);
```

- Есть массивы:

```
let a = array<f32, 5>(0.0, 1.0, 2.0, 3.0, 4.0);
```

- Константы (известные на момент компиляции):

```
const float PI = 3.141592;
```

- Ветвление: `if` (condition) { ... } `else` { ... }

- Ветвление: `if (condition) { ... } else { ... }`
- Тернарный if: `select(if_false, if_true, condition);`

Язык описания шейдеров WGSL

- Ветвление: `if (condition) { ... } else { ... }`
- Тернарный if: `select(if_false, if_true, condition);`
- Циклы: `for (var i = 0; i < 10; ++i) { ... }`
 - Бесконечные циклы могут проявиться по-разному, вплоть до краша всей системы :)

Язык описания шейдеров WGSL

- Ветвление: `if (condition) { ... } else { ... }`
- Тернарный if: `select(if_false, if_true, condition);`
- Циклы: `for (var i = 0; i < 10; ++i) { ... }`
 - Бесконечные циклы могут проявиться по-разному, вплоть до крэша всей системы :)
- Функции:

```
fn diffuse(n: vec3f, l: vec3f) -> f32 {  
    return max(0.0, dot(n, l));  
}
```

Язык описания шейдеров WGSL

- Ветвление: `if (condition) { ... } else { ... }`
- Тернарный if: `select(if_false, if_true, condition);`
- Циклы: `for (var i = 0; i < 10; ++i) { ... }`
 - Бесконечные циклы могут проявиться по-разному, вплоть до крэша всей системы :)
- Функции:

```
fn diffuse(n: vec3f, l: vec3f) -> f32 {  
    return max(0.0, dot(n, l));  
}
```

- Могут вызывать другие функции

Язык описания шейдеров WGSL

- Ветвление: `if (condition) { ... } else { ... }`
- Тернарный if: `select(if_false, if_true, condition);`
- Циклы: `for (var i = 0; i < 10; ++i) { ... }`
 - Бесконечные циклы могут проявиться по-разному, вплоть до крэша всей системы :)
- Функции:

```
fn diffuse(n: vec3f, l: vec3f) -> f32 {  
    return max(0.0, dot(n, l));  
}
```

- Могут вызывать другие функции
- Рекурсия запрещена

- Про условные конструкции в шейдерах ходит очень много мифов, вот TL;DR:

Язык описания шейдеров WGSL

- Про условные конструкции в шейдерах ходит очень много мифов, вот TL;DR:
- Старые GPU действительно плохо умели их обрабатывать из-за *wave divergence*: разным ядрам одного wavefront'a приходится выполнять разные операции

- Про условные конструкции в шейдерах ходит очень много мифов, вот TL;DR:
- Старые GPU действительно плохо умели их обрабатывать из-за *wave divergence*: разным ядрам одного *wavefront*'а приходится выполнять разные операции
- Современные GPU хорошо обрабатывают условия, если их результат одинаковый в рамках одного *wavefront*'а (близкие по номеру вершины, близкие на экране пиксели)

Язык описания шейдеров WGSL

- Про условные конструкции в шейдерах ходит очень много мифов, вот TL;DR:
- Старые GPU действительно плохо умели их обрабатывать из-за *wave divergence*: разным ядрам одного *wavefront*'а приходится выполнять разные операции
- Современные GPU хорошо обрабатывают условия, если их результат одинаковый в рамках одного *wavefront*'а (близкие по номеру вершины, близкие на экране пиксели)
- Например, когда условие зависит только от *uniform* или *immediate* переменных (т.н. *uniform control flow*), его результат один для всех вызовов шейдера (в рамках одной команды рисования)

- Тьюториал: `google.github.io/tour-of-wgsl`
- Тьюториал: `webgpufundamentals.org/webgpu/lessons/webgpu-wgsl.html`
- Книжка-учебник с большим количеством примеров сложных шейдеров: **The Book of Shaders**
- Онлайн-редактор шейдеров: `shadertoy.com`

Четырёхмерные векторы?

- WGSL поддерживает операции с матрицами и векторами размерностей вплоть до 4
- `@builtin(position)` имеет 4 компоненты

Четырёхмерные векторы?

- WGSL поддерживает операции с матрицами и векторами размерностей вплоть до 4
- `@builtin(position)` имеет 4 компоненты
- Мы хотим рисовать двухмерные и трёхмерные объекты, зачем четвёртая координата?

Четырёхмерные векторы?

- WGSL поддерживает операции с матрицами и векторами размерностей вплоть до 4
- `@builtin(position)` имеет 4 компоненты
- Мы хотим рисовать двухмерные и трёхмерные объекты, зачем четвёртая координата?
- Три причины:

Четырёхмерные векторы?

- WGSL поддерживает операции с матрицами и векторами размерностей вплоть до 4
- `@builtin(position)` имеет 4 компоненты
- Мы хотим рисовать двухмерные и трёхмерные объекты, зачем четвёртая координата?
- Три причины:
 - Для некоторых *типов данных*: кватернионы, цвета (rgba)

Четырёхмерные векторы?

- WGSL поддерживает операции с матрицами и векторами размерностей вплоть до 4
- `@builtin(position)` имеет 4 компоненты
- Мы хотим рисовать двухмерные и трёхмерные объекты, зачем четвёртая координата?
- Три причины:
 - Для некоторых *типов данных*: кватернионы, цвета (rgba)
 - Для *перспективной проекции* (4ая лекция)

Четырёхмерные векторы?

- WGSL поддерживает операции с матрицами и векторами размерностей вплоть до 4
- `@builtin(position)` имеет 4 компоненты
- Мы хотим рисовать двухмерные и трёхмерные объекты, зачем четвёртая координата?
- Три причины:
 - Для некоторых *типов данных*: кватернионы, цвета (rgba)
 - Для *перспективной проекции* (4ая лекция)
 - Для *аффинных преобразований*

- Векторные (линейные) пространства – круто: большая теория, эффективные операции

- Векторные (линейные) пространства – круто: большая теория, эффективные операции
- Не очень подходит для того, чтобы моделировать пространство

- Векторные (линейные) пространства – круто: большая теория, эффективные операции
- Не очень подходит для того, чтобы моделировать пространство
 - Где в пространстве точка, соответствующая нулевому вектору?

- Векторные (линейные) пространства – круто: большая теория, эффективные операции
- Не очень подходит для того, чтобы моделировать пространство
 - Где в пространстве точка, соответствующая нулевому вектору?
 - Что такое сумма двух точек?

- Векторные (линейные) пространства – круто: большая теория, эффективные операции
- Не очень подходит для того, чтобы моделировать пространство
 - Где в пространстве точка, соответствующая нулевому вектору?
 - Что такое сумма двух точек?
- Мысль: точки $\neq$ векторы

- Аффинное пространство: векторное пространство, в котором “забыли” нулевой вектор

- Аффинное пространство: векторное пространство, в котором “забыли” нулевой вектор
- Формальнее, аффинное пространство $\mathbb{A}$ над векторным пространством $\mathbb{V}$ – это:

Аффинное пространство

- Аффинное пространство: векторное пространство, в котором “забыли” нулевой вектор
- Формальнее, аффинное пространство $\mathbb{A}$ над векторным пространством $\mathbb{V}$ – это:
 - Множество точек $\mathbb{A}$

Аффинное пространство

- Аффинное пространство: векторное пространство, в котором “забыли” нулевой вектор
- Формальнее, аффинное пространство $\mathbb{A}$ над векторным пространством $\mathbb{V}$ – это:
 - Множество точек $\mathbb{A}$
 - Операция “точка + вектор”: $\oplus : \mathbb{A} \times \mathbb{V} \rightarrow \mathbb{A}$

Аффинное пространство

- Аффинное пространство: векторное пространство, в котором “забыли” нулевой вектор
- Формальнее, аффинное пространство $\mathbb{A}$ над векторным пространством $\mathbb{V}$ – это:
 - Множество точек $\mathbb{A}$
 - Операция “точка + вектор”: $\oplus : \mathbb{A} \times \mathbb{V} \rightarrow \mathbb{A}$
 - Операция “точка - точка”: $\ominus : \mathbb{A} \times \mathbb{A} \rightarrow \mathbb{V}$

Аффинное пространство

- Аффинное пространство: векторное пространство, в котором “забыли” нулевой вектор
- Формальнее, аффинное пространство $\mathbb{A}$ над векторным пространством $\mathbb{V}$ – это:
 - Множество точек $\mathbb{A}$
 - Операция “точка + вектор”: $\oplus : \mathbb{A} \times \mathbb{V} \rightarrow \mathbb{A}$
 - Операция “точка - точка”: $\ominus : \mathbb{A} \times \mathbb{A} \rightarrow \mathbb{V}$
 - Аксиомы, связывающие их с операциями векторного пространства, например $(p \oplus v_1) \oplus v_2 = p \oplus (v_1 + v_2)$

Аффинное пространство

- Аффинное пространство: векторное пространство, в котором “забыли” нулевой вектор
- Формальнее, аффинное пространство $\mathbb{A}$ над векторным пространством $\mathbb{V}$ – это:
 - Множество точек $\mathbb{A}$
 - Операция “точка + вектор”: $\oplus : \mathbb{A} \times \mathbb{V} \rightarrow \mathbb{A}$
 - Операция “точка - точка”: $\ominus : \mathbb{A} \times \mathbb{A} \rightarrow \mathbb{V}$
 - Аксиомы, связывающие их с операциями векторного пространства, например $(p \oplus v_1) \oplus v_2 = p \oplus (v_1 + v_2)$
- Любую точку $p_0 \in \mathbb{A}$ можно принять за начало координат
- Это даёт биекцию $\mathbb{A} \rightarrow \mathbb{V}$ по формуле $p \mapsto p - p_0$

- Множество решений уравнения $Lx = 0$ в векторном пространстве – *векторное подпространство* $\ker L$

- Множество решений уравнения $Lx = 0$ в векторном пространстве – *векторное подпространство* $\ker L$
- Множество решений уравнения $Lx = y$ в векторном пространстве – **не** векторное подпространство

- Множество решений уравнения $Lx = 0$ в векторном пространстве – *векторное подпространство* $\ker L$
- Множество решений уравнения $Lx = y$ в векторном пространстве – **не** векторное подпространство
- Множество решений уравнения $Lx = y$ в векторном пространстве – *аффинное пространство* над $\ker L$

Аффинные комбинации

- Пусть есть набор точек $p_1, \dots, p_N$ и набор коэффициентов $\lambda_1, \dots, \lambda_N$

Аффинные комбинации

- Пусть есть набор точек $p_1, \dots, p_N$ и набор коэффициентов $\lambda_1, \dots, \lambda_N$
- Выберем начало координат p_0 и определим
$$\sum \lambda_i p_i = p_0 + \sum \lambda_i (p_i - p_0) \in \mathbb{A}$$

Аффинные комбинации

- Пусть есть набор точек $p_1, \dots, p_N$ и набор коэффициентов $\lambda_1, \dots, \lambda_N$
- Выберем начало координат p_0 и определим
$$\sum \lambda_i p_i = p_0 + \sum \lambda_i (p_i - p_0) \in \mathbb{A}$$
- Теорема: если $\sum \lambda_i = 1$, то результат (точка) не зависит от выбора p_0

Аффинные комбинации

- Пусть есть набор точек $p_1, \dots, p_N$ и набор коэффициентов $\lambda_1, \dots, \lambda_N$
- Выберем начало координат p_0 и определим
$$\sum \lambda_i p_i = p_0 + \sum \lambda_i (p_i - p_0) \in \mathbb{A}$$
- Теорема: если $\sum \lambda_i = 1$, то результат (точка) не зависит от выбора p_0
- В этом случае $q = \sum \lambda_i p_i$ называется *аффинной комбинацией* точек p_i с коэффициентами λ_i

Аффинные комбинации

- Пусть есть набор точек $p_1, \dots, p_N$ и набор коэффициентов $\lambda_1, \dots, \lambda_N$
- Выберем начало координат p_0 и определим
$$\sum \lambda_i p_i = p_0 + \sum \lambda_i (p_i - p_0) \in \mathbb{A}$$
- Теорема: если $\sum \lambda_i = 1$, то результат (точка) не зависит от выбора p_0
- В этом случае $q = \sum \lambda_i p_i$ называется *аффинной комбинацией* точек p_i с коэффициентами λ_i
- Коэффициенты λ_i называются *барицентрическими координатами* точки q

Аффинные комбинации

- Пусть есть набор точек $p_1, \dots, p_N$ и набор коэффициентов $\lambda_1, \dots, \lambda_N$
- Выберем начало координат p_0 и определим
$$\sum \lambda_i p_i = p_0 + \sum \lambda_i (p_i - p_0) \in \mathbb{A}$$
- Теорема: если $\sum \lambda_i = 1$, то результат (точка) не зависит от выбора p_0
- В этом случае $q = \sum \lambda_i p_i$ называется *аффинной комбинацией* точек p_i с коэффициентами λ_i
- Коэффициенты λ_i называются *барицентрическими координатами* точки q
- Поиск λ_i по известным q и p_i сводится к решению линейной системы уравнений (квадратная только если $N = D + 1$, где D – размерность пространства)

Аффинные комбинации

- Пусть есть набор точек $p_1, \dots, p_N$ и набор коэффициентов $\lambda_1, \dots, \lambda_N$
- Выберем начало координат p_0 и определим
$$\sum \lambda_i p_i = p_0 + \sum \lambda_i (p_i - p_0) \in \mathbb{A}$$
- Теорема: если $\sum \lambda_i = 1$, то результат (точка) не зависит от выбора p_0
- В этом случае $q = \sum \lambda_i p_i$ называется *аффинной комбинацией* точек p_i с коэффициентами λ_i
- Коэффициенты λ_i называются *барицентрическими координатами* точки q
- Поиск λ_i по известным q и p_i сводится к решению линейной системы уравнений (квадратная только если $N = D + 1$, где D – размерность пространства)
- Аффинная комбинация пустого множества точек не определена

Аффинные комбинации: пример



Аффинные комбинации: пример

$$\lambda_1 = 1$$

$$\lambda_2 = 0$$



p_1



p_2

Аффинные комбинации: пример

$$\lambda_1 = 0$$

$$\lambda_2 = 1$$



p_1

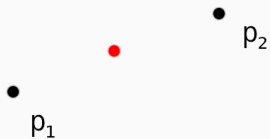


p_2

Аффинные комбинации: пример

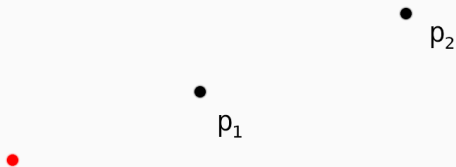
$$\lambda_1 = 0.5$$

$$\lambda_2 = 0.5$$



Аффинные комбинации: пример

$$\begin{aligned}\lambda_1 &= 2 \\ \lambda_2 &= -1\end{aligned}$$



- Пусть есть набор точек $p_1, \dots, p_N$ и набор коэффициентов $\lambda_1, \dots, \lambda_N$

- Пусть есть набор точек $p_1, \dots, p_N$ и набор коэффициентов $\lambda_1, \dots, \lambda_N$
- Выберем начало координат p_0 и определим
$$\sum \lambda_i p_i = \sum \lambda_i (p_i - p_0) \in \mathbb{V}$$

- Пусть есть набор точек $p_1, \dots, p_N$ и набор коэффициентов $\lambda_1, \dots, \lambda_N$
- Выберем начало координат p_0 и определим
$$\sum \lambda_i p_i = \sum \lambda_i (p_i - p_0) \in \mathbb{V}$$
- Теорема: если $\sum \lambda_i = 0$, то результат (вектор) не зависит от выбора p_0

- Пусть есть набор точек $p_1, \dots, p_N$

- Пусть есть набор точек $p_1, \dots, p_N$
- Их *аффинная оболочка* – множество всех возможных аффинных комбинаций, т.е. множество $\sum \lambda_i p_i$ для всех возможных λ_i таких, что $\sum \lambda_i = 1$

- Пусть есть набор точек $p_1, \dots, p_N$
- Их *аффинная оболочка* – множество всех возможных аффинных комбинаций, т.е. множество $\sum \lambda_i p_i$ для всех возможных λ_i таких, что $\sum \lambda_i = 1$
- Аффинная оболочка точки – сама *точка*

Аффинная оболочка

- Пусть есть набор точек $p_1, \dots, p_N$
- Их *аффинная оболочка* – множество всех возможных аффинных комбинаций, т.е. множество $\sum \lambda_i p_i$ для всех возможных λ_i таких, что $\sum \lambda_i = 1$
- Аффинная оболочка точки – сама *точка*
- Аффинная оболочка двух точек – содержащая их *прямая*

Аффинная оболочка

- Пусть есть набор точек $p_1, \dots, p_N$
- Их *аффинная оболочка* – множество всех возможных аффинных комбинаций, т.е. множество $\sum \lambda_i p_i$ для всех возможных λ_i таких, что $\sum \lambda_i = 1$
- Аффинная оболочка точки – сама *точка*
- Аффинная оболочка двух точек – содержащая их *прямая*
- Аффинная оболочка трёх точек – содержащая их *плоскость*

Аффинная оболочка

- Пусть есть набор точек $p_1, \dots, p_N$
- Их *аффинная оболочка* – множество всех возможных аффинных комбинаций, т.е. множество $\sum \lambda_i p_i$ для всех возможных λ_i таких, что $\sum \lambda_i = 1$
- Аффинная оболочка точки – сама *точка*
- Аффинная оболочка двух точек – содержащая их *прямая*
- Аффинная оболочка трёх точек – содержащая их *плоскость*
- И т.д.

- Линейная интерполяция = интерполяция, использующая барицентрические координаты как коэффициенты

Линейная интерполяция

- Линейная интерполяция = интерполяция, использующая барицентрические координаты как коэффициенты
- Пусть есть точки p_i и точка $q = \sum \lambda_i p_i$

Линейная интерполяция

- Линейная интерполяция = интерполяция, использующая барицентрические координаты как коэффициенты
- Пусть есть точки p_i и точка $q = \sum \lambda_i p_i$
- Пусть в точках p_i задано значение некоторой величины f_i

Линейная интерполяция

- Линейная интерполяция = интерполяция, использующая барицентрические координаты как коэффициенты
- Пусть есть точки p_i и точка $q = \sum \lambda_i p_i$
- Пусть в точках p_i задано значение некоторой величины f_i
- Линейная интерполяция величины f в точке q – значение $\sum \lambda_i f_i$

Линейная интерполяция

- Линейная интерполяция = интерполяция, использующая барицентрические координаты как коэффициенты
- Пусть есть точки p_i и точка $q = \sum \lambda_i p_i$
- Пусть в точках p_i задано значение некоторой величины f_i
- Линейная интерполяция величины f в точке q – значение $\sum \lambda_i f_i$
- Для этого нужно вычислять λ_i по известным p_i и q , что сводится к системе линейных уравнений

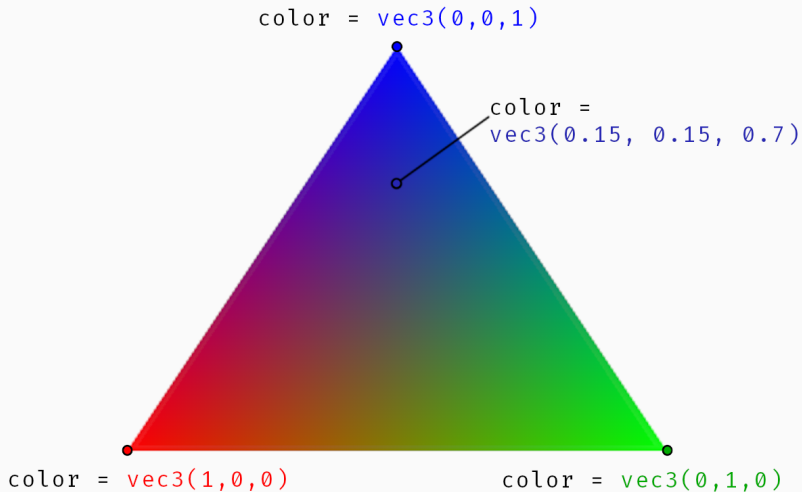
Линейная интерполяция

- Линейная интерполяция = интерполяция, использующая барицентрические координаты как коэффициенты
- Пусть есть точки p_i и точка $q = \sum \lambda_i p_i$
- Пусть в точках p_i задано значение некоторой величины f_i
- Линейная интерполяция величины f в точке q – значение $\sum \lambda_i f_i$
- Для этого нужно вычислять λ_i по известным p_i и q , что сводится к системе линейных уравнений
- **Ровно это** происходит при интерполяции значений, переданных из вершинного шейдера во фрагментный

Линейная интерполяция

- Линейная интерполяция = интерполяция, использующая барицентрические координаты как коэффициенты
- Пусть есть точки p_i и точка $q = \sum \lambda_i p_i$
- Пусть в точках p_i задано значение некоторой величины f_i
- Линейная интерполяция величины f в точке q – значение $\sum \lambda_i f_i$
- Для этого нужно вычислять λ_i по известным p_i и q , что сводится к системе линейных уравнений
- **Ровно это** происходит при интерполяции значений, переданных из вершинного шейдера во фрагментный (с точностью до перспективной проекции, о чём мы поговорим позже)

Линейная интерполяция цветов



- Выпуклая комбинация: аффинная комбинация, в которой все коэффициенты неотрицательны $\lambda_i \geq 0$

- Выпуклая комбинация: аффинная комбинация, в которой все коэффициенты неотрицательны $\lambda_i \geq 0$
 - Вместе с $\sum \lambda_i = 1$ это означает, что $\lambda_i \in [0, 1]$

Выпуклые оболочки

- Выпуклая комбинация: аффинная комбинация, в которой все коэффициенты неотрицательны $\lambda_i \geq 0$
 - Вместе с $\sum \lambda_i = 1$ это означает, что $\lambda_i \in [0, 1]$
- Множество всех выпуклых комбинаций набора точек = *выпуклая оболочка*

Выпуклые оболочки

- Выпуклая комбинация: аффинная комбинация, в которой все коэффициенты неотрицательны $\lambda_i \geq 0$
 - Вместе с $\sum \lambda_i = 1$ это означает, что $\lambda_i \in [0, 1]$
- Множество всех выпуклых комбинаций набора точек = *выпуклая оболочка*
- Выпуклая оболочка точки – сама *точка*

Выпуклые оболочки

- Выпуклая комбинация: аффинная комбинация, в которой все коэффициенты неотрицательны $\lambda_i \geq 0$
 - Вместе с $\sum \lambda_i = 1$ это означает, что $\lambda_i \in [0, 1]$
- Множество всех выпуклых комбинаций набора точек = *выпуклая оболочка*
- Выпуклая оболочка точки – сама *точка*
- Выпуклая оболочка двух точек – *отрезок* между этими точками

Выпуклые оболочки

- Выпуклая комбинация: аффинная комбинация, в которой все коэффициенты неотрицательны $\lambda_i \geq 0$
 - Вместе с $\sum \lambda_i = 1$ это означает, что $\lambda_i \in [0, 1]$
- Множество всех выпуклых комбинаций набора точек = *выпуклая оболочка*
- Выпуклая оболочка точки – сама *точка*
- Выпуклая оболочка двух точек – *отрезок* между этими точками
- Выпуклая оболочка трёх точек – *треугольник* с этими точками в качестве вершин

Выпуклые оболочки

- Выпуклая комбинация: аффинная комбинация, в которой все коэффициенты неотрицательны $\lambda_i \geq 0$
 - Вместе с $\sum \lambda_i = 1$ это означает, что $\lambda_i \in [0, 1]$
- Множество всех выпуклых комбинаций набора точек = *выпуклая оболочка*
- Выпуклая оболочка точки – сама *точка*
- Выпуклая оболочка двух точек – *отрезок* между этими точками
- Выпуклая оболочка трёх точек – *треугольник* с этими точками в качестве вершин
- Выпуклая оболочка четырёх точек – *тетраэдр* с этими точками в качестве вершин

Выпуклые оболочки

- Выпуклая комбинация: аффинная комбинация, в которой все коэффициенты неотрицательны $\lambda_i \geq 0$
 - Вместе с $\sum \lambda_i = 1$ это означает, что $\lambda_i \in [0, 1]$
- Множество всех выпуклых комбинаций набора точек = *выпуклая оболочка*
- Выпуклая оболочка точки – сама *точка*
- Выпуклая оболочка двух точек – *отрезок* между этими точками
- Выпуклая оболочка трёх точек – *треугольник* с этими точками в качестве вершин
- Выпуклая оболочка четырёх точек – *тетраэдр* с этими точками в качестве вершин
- И т.д.

- При растеризации центры пикселей попадают внутрь треугольника

Выпуклые оболочки

- При растеризации центры пикселей попадают внутрь треугольника
- $\Rightarrow$ Пиксели лежат в *выпуклой оболочке* исходных вершин

Выпуклые оболочки

- При растеризации центры пикселей попадают внутрь треугольника
- $\Rightarrow$ Пиксели лежат в *выпуклой оболочке* исходных вершин
- $\Rightarrow$ Коэффициенты интерполяции всегда в диапазоне $\lambda_i \in [0, 1]$

Выпуклые оболочки

- При растеризации центры пикселей попадают внутрь треугольника
- $\Rightarrow$ Пиксели лежат в *выпуклой оболочке* исходных вершин
- $\Rightarrow$ Коэффициенты интерполяции всегда в диапазоне $\lambda_i \in [0, 1]$
- $\Rightarrow$ Проинтерполированные значения лежат в выпуклой оболочке значений в вершинах
 - Например, если *out*-значения величины в вершинах были 1.5, 0.3 и 5.7, то *in*-значения в пикселях будут в диапазоне $[0.3, 5.7]$ (с точностью до *floating-point* ошибок)

Выпуклые оболочки

- При растеризации центры пикселей попадают внутрь треугольника
- $\Rightarrow$ Пиксели лежат в *выпуклой оболочке* исходных вершин
- $\Rightarrow$ Коэффициенты интерполяции всегда в диапазоне $\lambda_i \in [0, 1]$
- $\Rightarrow$ Проинтерполированные значения лежат в выпуклой оболочке значений в вершинах
 - Например, если *out*-значения величины в вершинах были 1.5, 0.3 и 5.7, то *in*-значения в пикселях будут в диапазоне $[0.3, 5.7]$ (с точностью до *floating-point* ошибок)
- N.B. Это может нарушаться при *мультисэмплинге*, о котором мы поговорим позднее

Аффинные преобразования

- Линейные преобразования – сохраняют *линейные комбинации* векторов: $S(\sum \lambda_i v_i) = \sum \lambda_i S v_i$

Аффинные преобразования

- Линейные преобразования – сохраняют *линейные комбинации* векторов: $S(\sum \lambda_i v_i) = \sum \lambda_i S v_i$
- Аффинные преобразования – сохраняют *аффинные комбинации* точек: $S(\sum \lambda_i p_i) = \sum \lambda_i S p_i$

Аффинные преобразования

- Линейные преобразования – сохраняют *линейные комбинации* векторов: $S(\sum \lambda_i v_i) = \sum \lambda_i S v_i$
- Аффинные преобразования – сохраняют *аффинные комбинации* точек: $S(\sum \lambda_i p_i) = \sum \lambda_i S p_i$
- Если выбрано начало координат p_0 , преобразование $S : \mathbb{A} \rightarrow \mathbb{A}$ можно понимать как преобразование соответствующего векторного пространства $S : \mathbb{V} \rightarrow \mathbb{V}$ по формуле $S(v) = S(p_0 + v) - p_0$

Аффинные преобразования

- Линейные преобразования – сохраняют *линейные комбинации* векторов: $S(\sum \lambda_i v_i) = \sum \lambda_i S v_i$
- Аффинные преобразования – сохраняют *аффинные комбинации* точек: $S(\sum \lambda_i p_i) = \sum \lambda_i S p_i$
- Если выбрано начало координат p_0 , преобразование $S : \mathbb{A} \rightarrow \mathbb{A}$ можно понимать как преобразование соответствующего векторного пространства $S : \mathbb{V} \rightarrow \mathbb{V}$ по формуле $S(v) = S(p_0 + v) - p_0$
- Можно показать, что в таком виде любое аффинное преобразование это линейное преобразование + сдвиг на фиксированный вектор:
 $S(v) = Av + b$ где
 - A – линейное преобразование пространства $\mathbb{V}$
 - b – вектор из пространства $\mathbb{V}$

Аффинные преобразования

- В коде аффинное преобразование пространства размерности N можно представлять как *пару* (A, b) из матрицы $N \times N$ и вектора размерности N

Аффинные преобразования

- В коде аффинное преобразование пространства размерности N можно представлять как *пару* (A, b) из матрицы $N \times N$ и вектора размерности N
- Пара (A, b) действует на точку/вектор v по формуле $(A, b) \cdot v = Av + b$

Аффинные преобразования

- В коде аффинное преобразование пространства размерности N можно представлять как пару (A, b) из матрицы $N \times N$ и вектора размерности N
- Пара (A, b) действует на точку/вектор v по формуле $(A, b) \cdot v = Av + b$
- Композиция преобразований:
$$(A_1, b_1) \cdot (A_2, b_2) \cdot v = (A_1, b_1) \cdot (A_2v + b_2) = A_1(A_2v + b_2) + b_1 = (A_1A_2)v + (A_1b_2 + b_1) = (A_1A_2, A_1b_2 + b_1) \cdot v$$

Аффинные преобразования

- В коде аффинное преобразование пространства размерности N можно представлять как пару (A, b) из матрицы $N \times N$ и вектора размерности N
- Пара (A, b) действует на точку/вектор v по формуле $(A, b) \cdot v = Av + b$
- Композиция преобразований:
$$(A_1, b_1) \cdot (A_2, b_2) \cdot v = (A_1, b_1) \cdot (A_2v + b_2) = A_1(A_2v + b_2) + b_1 = (A_1A_2)v + (A_1b_2 + b_1) = (A_1A_2, A_1b_2 + b_1) \cdot v$$
- Тожественное преобразование: $(I, 0)$

Аффинные преобразования

- В коде аффинное преобразование пространства размерности N можно представлять как пару (A, b) из матрицы $N \times N$ и вектора размерности N
- Пара (A, b) действует на точку/вектор v по формуле $(A, b) \cdot v = Av + b$
- Композиция преобразований:
$$(A_1, b_1) \cdot (A_2, b_2) \cdot v = (A_1, b_1) \cdot (A_2v + b_2) = A_1(A_2v + b_2) + b_1 = (A_1A_2)v + (A_1b_2 + b_1) = (A_1A_2, A_1b_2 + b_1) \cdot v$$
- Тожественное преобразование: $(I, 0)$
- Обратное преобразование:
$$(A, b)^{-1} = (A^{-1}, -A^{-1}b)$$

- Повсеместно используются для задания положения, ориентации и размера объектов

- Повсеместно используются для задания положения, ориентации и размера объектов
- Векторная часть преобразования: положение 3D-объекта

- Повсеместно используются для задания положения, ориентации и размера объектов
- Векторная часть преобразования: положение 3D-объекта
- Матричная часть преобразования: вращение и размер объекта

- Повсеместно используются для задания положения, ориентации и размера объектов
- Векторная часть преобразования: положение 3D-объекта
- Матричная часть преобразования: вращение и размер объекта
- При чём тут четвёртая координата?

- Трюк: можно вложить *вектор* размерности N в векторное пространство размерности $N + 1$, добавив 0 в качестве последней координаты

- Трюк: можно вложить *вектор* размерности N в векторное пространство размерности $N + 1$, добавив 0 в качестве последней координаты
- Трюк: можно вложить *точку* размерности N в векторное пространство размерности $N + 1$, добавив 1 в качестве последней координаты

Однородные координаты

- Трюк: можно вложить *вектор* размерности N в векторное пространство размерности $N + 1$, добавив 0 в качестве последней координаты
- Трюк: можно вложить *точку* размерности N в векторное пространство размерности $N + 1$, добавив 1 в качестве последней координаты
- Такое представление точек и векторов называется *однородными координатами*

Однородные координаты

- Трюк: можно вложить *вектор* размерности N в векторное пространство размерности $N + 1$, добавив 0 в качестве последней координаты
- Трюк: можно вложить *точку* размерности N в векторное пространство размерности $N + 1$, добавив 1 в качестве последней координаты
- Такое представление точек и векторов называется *однородными координатами*
- Сопласуется с операциями на векторах и точках

Однородные координаты

- Трюк: можно вложить *вектор* размерности N в векторное пространство размерности $N + 1$, добавив 0 в качестве последней координаты
- Трюк: можно вложить *точку* размерности N в векторное пространство размерности $N + 1$, добавив 1 в качестве последней координаты
- Такое представление точек и векторов называется *однородными координатами*
- Сопласуется с операциями на векторах и точках
- Сопласуется с аффинными комбинациями

- Позволяет представить аффинное преобразование N -мерного пространства как матрицу $(N + 1) \times (N + 1)$:

$$\begin{pmatrix} A & b \\ 0 & 1 \end{pmatrix}$$

- Позволяет представить аффинное преобразование N -мерного пространства как матрицу $(N + 1) \times (N + 1)$:

$$\begin{pmatrix} A & b \\ 0 & 1 \end{pmatrix}$$

- Можно применять как к точкам, так и к векторам (векторы игнорируют сдвиг на b)

- Позволяет представить аффинное преобразование N -мерного пространства как матрицу $(N + 1) \times (N + 1)$:

$$\begin{pmatrix} A & b \\ 0 & 1 \end{pmatrix}$$

- Можно применять как к точкам, так и к векторам (векторы игнорируют сдвиг на b)
- Композиция преобразований = умножение матриц

Однородные координаты

- Позволяет представить аффинное преобразование N -мерного пространства как матрицу $(N + 1) \times (N + 1)$:

$$\begin{pmatrix} A & b \\ 0 & 1 \end{pmatrix}$$

- Можно применять как к точкам, так и к векторам (векторы игнорируют сдвиг на b)
- Композиция преобразований = умножение матриц
- $\Rightarrow$ Позволяет удобно комбинировать преобразования (напр. масштабирование + сдвиг + поворот + другой сдвиг)

Однородные координаты

- Позволяет представить аффинное преобразование N -мерного пространства как матрицу $(N + 1) \times (N + 1)$:

$$\begin{pmatrix} A & b \\ 0 & 1 \end{pmatrix}$$

- Можно применять как к точкам, так и к векторам (векторы игнорируют сдвиг на b)
- Композиция преобразований = умножение матриц
- $\Rightarrow$ Позволяет удобно комбинировать преобразования (напр. масштабирование + сдвиг + поворот + другой сдвиг)
- $\Rightarrow$ Позволяет собрать положение и ориентацию камеры и её проекцию в одну матрицу (обсудим на 4ой лекции)

Точка в однородных координатах

$$\begin{pmatrix} p_1 \\ p_2 \\ p_3 \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ 0 \end{pmatrix}$$

Аффинное преобразование в однородных координатах

$$\begin{pmatrix} A_{1,1} & A_{1,2} & A_{1,3} & b_1 \\ A_{2,1} & A_{2,2} & A_{2,3} & b_2 \\ A_{3,1} & A_{3,2} & A_{3,3} & b_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Матрица сдвига на фиксированный вектор

$$\begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix} = \begin{pmatrix} p_x + t_x \\ p_y + t_y \\ p_z + t_z \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \\ v_z \\ 0 \end{pmatrix} = \begin{pmatrix} v_x \\ v_y \\ v_z \\ 0 \end{pmatrix}$$

Матрица изотропного масштабирования

$$\begin{pmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix} = \begin{pmatrix} sp_x \\ sp_y \\ sp_z \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \\ v_z \\ 0 \end{pmatrix} = \begin{pmatrix} sv_x \\ sv_y \\ sv_z \\ 0 \end{pmatrix}$$

Матрица поворота на угол θ в плоскости XU

$$\begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix} = \begin{pmatrix} p_x \cos \theta - p_y \sin \theta \\ p_x \sin \theta + p_y \cos \theta \\ p_z \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \\ v_z \\ 0 \end{pmatrix} = \begin{pmatrix} v_x \cos \theta - v_y \sin \theta \\ v_x \sin \theta + v_y \cos \theta \\ v_z \\ 0 \end{pmatrix}$$

- learnopengl.com/Getting-started/Transformations
- open.gl/transformations
- en.wikipedia.org/wiki/Affine_space
- en.wikipedia.org/wiki/Affine_transformation